

AULA 02

Meu Primeiro Repositório e Linha do Tempo

Aprenda na prática a realizar o versionamento com Git, monitorando alterações de arquivos e criando um histórico seguro de commits para seu projeto.

Objetivo da Aula

- ✓ **Criar Repositório Local:** Criar o seu primeiro repositório Git local do absoluto zero.
- ✓ **Compreender a Staging Area:** Entender a área de preparação de arquivos antes do salvamento.
- ✓ **Configurar Identidade:** Definir o nome de autor e e-mail no terminal Git.
- ✓ **Registrar o Commit Inicial:** Gravar a primeira versão estável na linha do tempo.
- ✓ **Sincronizar com GitHub:** Conectar o projeto à nuvem e realizar a publicação do código.

1. Introdução Teórica



O Repositório

É a pasta do seu projeto monitorada pelo Git. É onde o sistema guarda todo o histórico de alterações e modificações feitas ao longo do tempo.



A Linha do Tempo

É construída através de "pontos de salvamento" chamados **Commits**. Cada commit funciona como uma foto exata do estado do código naquele momento.

⌚ Viagem no Tempo: Se cometer um erro no futuro, você pode facilmente restaurar o código para uma versão anterior que estava funcionando perfeitamente.

Passo 1: Criando a Pasta e Abrindo o Terminal

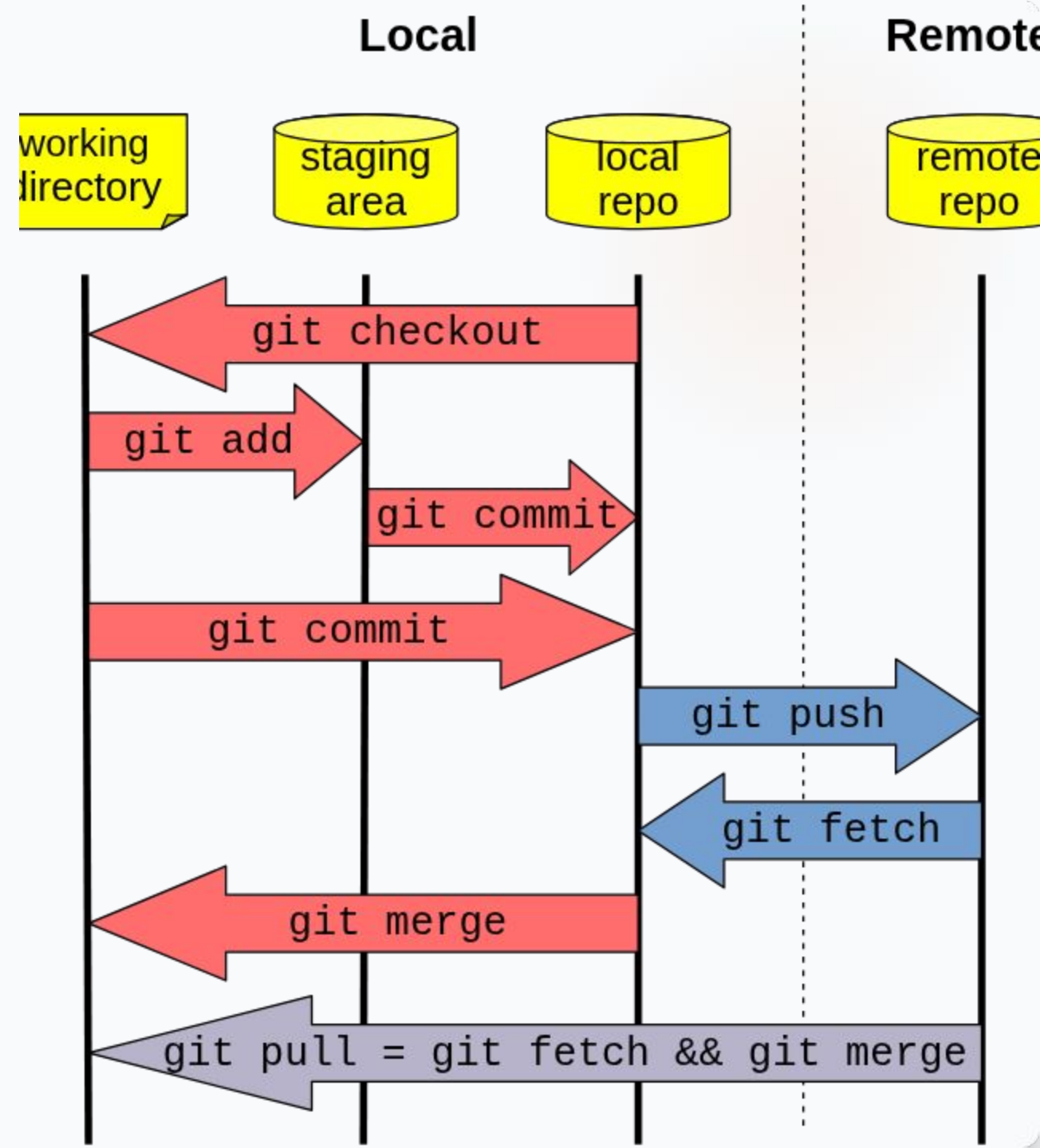
-  **Criar Pasta:** Escolha um local no seu computador e crie uma nova pasta chamada aula-1.
-  **Abrir Terminal:** Entre na pasta criada, clique com o botão direito em um espaço vazio e selecione a opção "**Abrir no Terminal**" (ou "Git Bash Here").
-  Isso abrirá o terminal diretamente apontado para o diretório do seu projeto.

Passo 2: Inicializando o Repositório

Para dizer ao Git que ele deve começar a monitorar a pasta, execute o comando:

```
git init
```

Por trás dos panos: O Git cria uma pasta oculta chamada `.git` onde guarda as configurações de versionamento.



Passo 2: Entendendo a Pasta .git

-  Para visualizar a pasta oculta .git, ative a opção "Itens ocultos" no gerenciador de arquivos do sistema operacional.
-  A pasta contém todos os objetos, referências, logs de histórico e configurações internas do repositório local.

⚠️ AVISO IMPORTANTE: Nunca modifique ou apague os arquivos da pasta .git manualmente! Isso destruirá todo o histórico de versionamento do projeto.

Passo 3: Criando o Primeiro Arquivo

Para testarmos o versionamento na prática, vamos simular um arquivo de código:

 Dentro da pasta do projeto, crie um novo arquivo de texto chamado `meu_codigo.txt`.

 Abra o arquivo e escreva o texto simples:

```
Olá, meu nome é [Seu Nome]
```

Salve e feche o arquivo após a edição.

Passo 4: Verificando o Estado (git status)

Pergunte ao Git qual é a situação atual do seu projeto executando:

```
git status
```

Análise do Feedback do Git:

O terminal exibirá um relatório informando que não existem commits feitos ainda e mostrará o arquivo meu_codigo.txt em vermelho na seção "**untracked files**" (não monitorados).

Significado: O arquivo existe na pasta, mas o Git ainda não o adicionou ao controle de versão.

Passo 5: Preparando Arquivos (git add)

Antes de salvar uma versão definitiva, precisamos colocar o arquivo na **área de preparação (staging area)**.

Arquivo Específico

Para preparar apenas um arquivo determinado:

```
git add "meu_codigo.txt"
```

Múltiplos Arquivos

Para preparar todos os arquivos alterados de uma vez:

```
git add .
```

Passo 5: Verificando a Staging Area

Após executar o comando git add, verifique novamente a situação do repositório:

```
git status
```

✔ **Resultado esperado:** O arquivo agora aparece na cor verde . Isso indica que foi preparado com sucesso e está pronto para ser salvo na linha do tempo!

Passo 6: Configurando sua Identidade

No primeiro commit no computador, o Git precisa saber quem é o autor da alteração. Configure e-mail e nome globalmente:

```
git config --global user.email "seu_email@exemplo.com" git config --global user.name "Seu Nome Real"
```

💡 **Nota Profissional:** Use exatamente o mesmo e-mail utilizado na sua conta do GitHub para que os commits fiquem vinculados ao seu perfil de desenvolvedor.

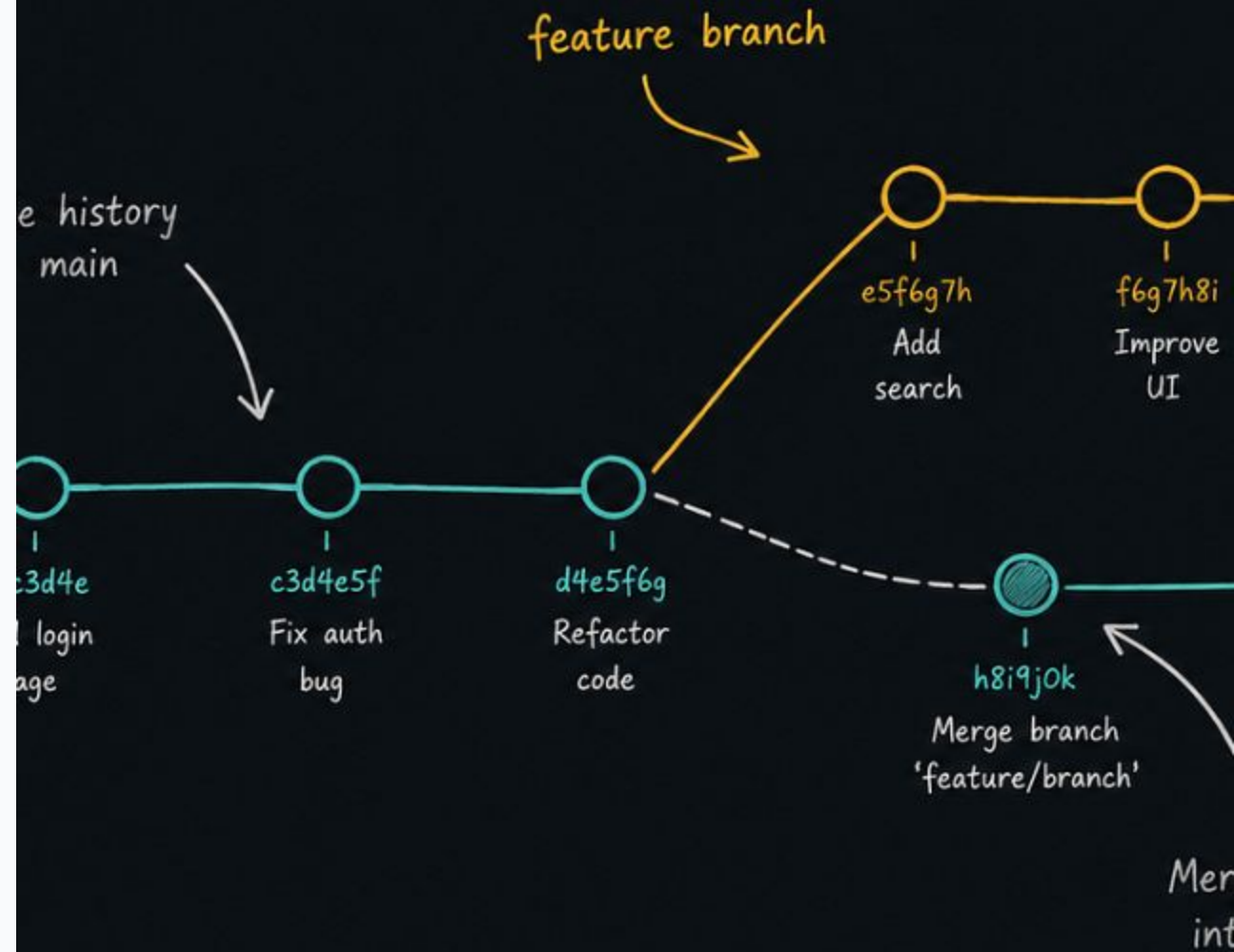
Passo 7: Criando o Commit

Com arquivos preparados e identidade pronta, vamos gravar a versão na linha do tempo:

```
git commit -m "Commit inicial"
```

O que é o **-m**? Significa "mensagem". Escreva sempre uma descrição clara entre aspas.

Ao rodar `git status` agora, o Git informará que não há nada pendente!



Passo 8: Conectando à Nuvem (1/3)

1. Criar o Repositório no GitHub

-  Acesse sua conta na plataforma GitHub.
-  Clique no botão "New" para criar um novo repositório.
-  Dê o nome de aula-1.
-  Clique em "Create repository".

 **Importante:** Deixe o repositório totalmente vazio (não adicione README, licença ou arquivo .gitignore neste momento).

Passo 8: Conectando à Nuvem (2/3)

2. Conectar o Repositório Local ao Remoto

No terminal da sua máquina local, cole e execute o comando fornecido pelo GitHub:

```
git remote add origin https://github.com/[SeuUsuario]/aula-1.git
```

O que significa?

Você está apresentando o servidor GitHub ao seu computador local. origin é o apelido padrão atribuído ao link remoto da internet.

Passo 8: Conectando à Nuvem (3/3)

3. Enviar o Código (git push)

```
git push -u origin master
```

(Nota: Se a branch principal for "main", use git push -u origin main)

O argumento -u (upstream) cria uma ligação definitiva. Nas próximas vezes, bastará digitar git push!



3. Desafio Prático: A Corrente do Git

Dinâmica em Grupo Colaborativa

Objetivo: Criar uma lista colaborativa no arquivo `alunos.txt` onde cada estudante adiciona seu nome e credenciais. Cada aluno fará o clone, configurará sua identidade local, fará a edição, registrará o commit e enviará ao servidor para o próximo colega continuar a corrente.

Fase 1: Preparação (Professor)

- 1 **Criar Repositório Público:** Crie no GitHub o repositório lista-turma.
- 2 **Arquivo Inicial:** Na máquina local, inicialize o projeto e crie o arquivo alunos.txt com o cabeçalho:

```
≡ LISTA DE ALUNOS - CURSO GIT E GITHUB ≡
```
- 3 **Enviar ao GitHub:** Faça o commit inicial e envie via git push.
- 4 **Liberar Acesso:** Em *Settings > Collaborators*, adicione os usuários GitHub de todos os alunos da equipe.

Fase 2: O Fluxo dos Alunos

Os alunos realizam as etapas de forma **sequencial** (um por vez). Quando um aluno termina, avisa o próximo da lista para repetir o processo.



Aluno 1

Clona, edita, envia e avisa o colega.



Aluno 2

Atualiza, edita, envia e passa o bastão.

Fluxo do Aluno: Passos 1 e 2

Passo 1: Clonar o Repositório

```
git clone https://github.com/[NOME_DO_LIDER]/lista-turma.git
```

Passo 2: Configurar Identidade Local

Configure nome e e-mail no repositório clonado:

```
git config user.email "seu_email_do_github@exemplo.com" git config user.name "Seu Nome Completo"
```

Fluxo do Aluno: Passo 3 (Atualização)

Passo 3: Atualizar o Arquivo Local

Antes de fazer alterações, garanta que possui a versão mais recente enviada pelo colega anterior:

```
git pull origin master
```

(Substitua master por main se for a branch principal do repositório)

 **Boa Prática Indispensável:** Executar o git pull previne conflitos de código e divergências de histórico no Git.

Fluxo do Aluno: Passo 4 (Edição)

Passo 4: Editar o Arquivo

Abra o arquivo alunos.txt em um editor de texto e adicione seus dados no final da lista no seguinte padrão:

```
- Nome: [Seu Nome Completo] | Usuário Git local: [Nome configurado no Passo 2] | E-mail: [Seu E-mail] | GitHub: @[SeuUsuario G  
itHub]
```

Salve o arquivo e feche o editor de texto.

Fluxo do Aluno: Passos 5 e 6

Passo 5: Verificar Status e Adicionar

```
git status git add alunos.txt
```

Passo 6: Fazer o Commit

```
git commit -m "[Seu usuário github]"
```

Fluxo do Aluno: Passos 7 e 8

Passo 7: Enviar para o GitHub

```
git push origin master
```

Passo 8: Passar o Bastão

- ✓ Acesse a página do repositório no GitHub para confirmar se os dados aparecem.
- 📢 Avise o próximo aluno da lista que a vez dele começou!

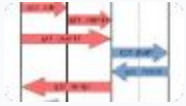


Parabéns!

Você aprendeu a criar seu primeiro repositório local, controlar o histórico de alterações e colaborar em equipe utilizando o GitHub.

Dúvidas?

Image Sources



<https://homes.cs.washington.edu/~mernst/advice/version-control-ops-local-vs-remote.svg>

Source: homes.cs.washington.edu



https://miro.medium.com/v2/resize:fit:1400/1*EpT9YHw-Au-Y1VsRWILVxw.png

Source: medium.com



<https://static0.makeuseofimages.com/wordpress/wp-content/uploads/wm/2026/03/close-up-of-git-pushing-files-to-notes-repository-on-github.JPG?w=1600&h=1200&fit=crop>

Source: www.makeuseof.com